

Propuestas de mejoras y nuevos módulos — ChambeaJobs

Enfoque: Candidato, Empresa y Monetización

Autor: Equipo ChambeaJobs Fecha: 2026-08-18

Índice 1. Introducción 2. Resumen técnico y hallazgos 3. Propuestas centradas en Candidato (detalladas) 4. Propuestas centradas en Empresa (detalladas) 5. Propuestas de Monetización y Paquetes (detalladas) 6. Herramientas y stack recomendados (cómo implementar) 7. Priorización y roadmap con pasos concretos 8. Checklist técnico y mapeo al código existente (con snippets) 9. Comandos y scripts útiles para desarrollo y despliegue 10. Estimaciones y recursos 11. Riesgos y mitigaciones 12. Anexos y referencias al código

1. Introducción Objetivo: presentar un plan amplio y accionable para mejorar ChambeaJobs centrado en tres ejes: experiencia del candidato, valor para empresas y monetización sostenible. Cada propuesta incluye por qué vale la pena, qué hay que cambiar en el código actual y pasos concretos para implementarla.
2. Resumen técnico y hallazgos
 - Arquitectura actual: ASP.NET Core MVC + EF Core + Identity; SignalR para tiempo real; patrón Repository/UnitOfWork; hosted services para background jobs; almacenamiento local para archivos (IFileStorageService -> LocalFileStorageService).
 - Módulos claves: Candidato, Empresa, Vacantes, Postulaciones, Favoritos, Admin (auditoría, reportes), Finanzas, CRM, Chat/Soporte/Chatbot, Evaluaciones. Se usan eventos de dominio para desacoplar comportamientos.
 - Puntos fuertes: seguridad (DataProtection + CSP), notificaciones en tiempo real, manejo de videoCV en límite de subida.
 - Oportunidades principales: mover storage a la nube, enriquecer perfiles candidatos, recomendaciones inteligentes, herramientas de gestión y analytics para empresas, flujos de pago y marketplace.
3. Propuestas centradas en Candidato (detalladas) 3.1 Perfil candidato enriquecido
 - Qué: perfil público “one-page” con campos estructurados (experiencia, educación, skills, certificaciones, links a portfolio, video-CV), visibilidad controlada y opción de compartir link público.
 - Por qué: reduce fricción y aumenta match; candidatos con perfil completo se postulan más y generan mayor confianza en empresas.
 - Cambios en el código:
 - Domain: nueva entidad PerfilCandidato (relacionada con ApplicationUser).
 - Application: DTOs y IPerfilCandidatoService (Get/Update/Publish/Unpublish).
 - Infrastructure: migraciones y repositorio para PerfilCandidato.
 - Web: vistas/partial views para edición y vista pública.
 - Pasos concretos:
 1. Crear entity ChambeaJobs.Domain/Entities/PerfilCandidato.cs con propiedades (Resumen, Skills[], VideoUrl, PortfolioLinks[]).
 2. Agregar DbSet en ApplicationDbContext.

3. Crear `IPerfilCandidatoService` + `PerfilCandidatoService` en `Application/Services`.
4. Rutas y views: `CandidatoController.Add/Update/ViewPublic`.

3.2 Video-CV + procesamiento - Qué: permitir grabar en el navegador y subir video; generar thumbnails y transcodificar a formatos web (MP4 H.264, WebM) y entregar vía CDN. - Herramientas recomendadas: FFmpeg (transcodificación), Azure Blob Storage o AWS S3 (almacenamiento), Azure Media Services (opcional) o Cloud Convert. - Cambios en el código: - `Infrastructure.Storage`: agregar `AzureBlobStorageService` / `S3StorageService`. - `Background worker`: `TranscoderBackgroundService` que escucha una cola (`RabbitMQ`/`Azure Service Bus`) y procesa nuevos videos con FFmpeg. - UI: subida en chunks (para mejorar fiabilidad) y endpoint para generar signed URLs. - Pasos concretos: 1. Añadir provider configurable en `appsettings.json` (`Storage:Provider`, `Container/ContainerName`, `ConnectionString`). 2. Implementar servicio de cola (`RabbitMQ/ServiceBus`) o usar hosted service para procesar localmente en fase inicial. 3. Usar FFmpeg en worker (puede ejecutarse en contenedor) para transcodificar y generar thumbnail. 4. Guardar metadatos (duración, size, thumbnails) en DB.

3.3 Parsing automático de CV y extracción de skills (NLP) - Qué: extraer skills, cargos y fechas desde documentos (PDF, DOCX, TXT) para guardar en estructuras normalizadas. - Herramientas: Python (`spaCy`, `Transformers`) o servicio SaaS (AWS Comprehend, Azure Text Analytics), o ML.NET para soluciones .NET nativas. - Integración técnica: - `IParserCvService` en `Infrastructure` que llama a un microservicio (`FastAPI/Flask`) o a un SDK externo. - Pipeline: subida -> almacenar -> enviar mensaje a cola -> servicio de parsing procesa -> escribe entidades en DB. - Pasos: 1. Implementar endpoint interno `/processor/cv` que acepte file path y devuelva JSON con skills/experiencia. 2. Mapear resultados a entidades en DB y vincular al `PerfilCandidato`.

3.4 Matching inteligente y recomendaciones - Qué: recomendaciones de vacantes personalizadas para cada candidato. - Fase 1 (rápida): heurística combinando matching de skills, relevancia temporal y afinidad por historial. - Fase 2: usar embeddings + vector DB (`Pinecone`, `Weaviate`, `Milvus`) para matching semántico. - Implementación: - Crear `IRecomendacionService` que exponga `GetRecommendedVacantes(userId, page)`. - Batch job que recalcula scores semanalmente y/o streaming con eventos (`PerfilActualizadoEvento`, `VacantePublicadaEvento`). - Requisitos de datos: normalizar skills (tags), guardar historial de clicks/postulaciones.

3.5 PWA y notificaciones push - Qué: convertir el front-end crítico a PWA; permitir notificaciones push para nuevas vacantes y respuestas. - Componentes: - `service-worker.js` - Endpoints para registrar token de suscripción (backend) - Servicio de envío push (`VAPID` via `web-push` library) - Pasos: 1. Añadir `manifest.json` y service worker en `wwwroot`. 2. Implementar endpoints `api/notifications/subscribe` y `api/notifications/send`. 3. Enviar notificaciones push desde C# usando `WebPush` (Lib: `Lib.Net.WebPush`) o desde un worker.

3.6 Evaluaciones y micro-certificaciones - Qué: pruebas cortas, autocalificadas y verificadas, con badges visibles en perfil. - Recomendado: comenzar con quizzes de selección múltiple y ejercicios autoevaluables; extender con code runner sandbox para pruebas técnicas (`Docker sandbox` o servicios como `Sphere Engine`). - Datos: almacenar resultados y emitir badges (verifiable badges con `Badgr/Open Badges` si se desea).

4. Propuestas centradas en Empresa (detalladas) 4.1 Dashboard de empresa y pipeline de candidatos

- Qué: panel con métricas: vistas, postulaciones, tasa de conversión, median time to hire, rendimiento de paquetes.

- Implementación:
 - Ampliar IFinanzasDashboardService y IReporteService para nuevos endpoints.
 - Crear vistas en Empresa area con filtros por vacante/fechas.
 - Exportar CSV/PDF desde backend (iText7 o DinkToPdf para server-side PDF si se necesita generar reportes).

4.2 Promoción de vacantes y campañas - Qué: opciones para destacar vacantes, promociones por targeting y newsletter. - Implementación: - Tabla Promotions con tipo, duración, coste, targeting. - Job que aplica boosts y ordena vacantes destacadas en las búsquedas.

4.3 API y Webhooks (integración ATS) - Qué: exponer API REST (v1) y webhooks para eventos importantes. - Seguridad: OAuth2 (IdentityServer4/ Duende) o API Keys + rate limiting. - Implementación técnica: - Crear proyecto ChambeaJobs.API (o Versioned API Controllers dentro de Web) con attribute [ApiController] y Swagger. - Registrar webhooks en tabla WebhookSubscriptions y enviar eventos con retries (usar Polly para HTTP retries y idempotency keys).

4.4 Candidate Screening y entrevistas en la plataforma - Qué: scheduling integrado y enlaces Jitsi para entrevistas en vivo; almacenamiento de grabaciones (si se habilita). - Implementación: - Endpoint para agendar entrevista (date/time) que notifique candidato y empresa. - Generar enlace Jitsi (puede ser un simple enlace a meet.jit.si/{random-room}). - Integración con calendar (Google Calendar API) opcional.

4.5 Employer CRM mejorado - Qué: notas, etiquetas, pipeline visual (kanban) por candidato. - Implementación: - Nuevas entidades: Tag, CandidateNote, PipelineStage. - UI: Kanban board (puede implementarse con JS libs como React + react-beautiful-dnd o alinearlos con Razor + Stimulus/Turbolinks para menor complejidad).

5. Propuestas de Monetización y Paquetes (detalladas) 5.1 Estructura de planes y reglas

- Plan Gratis: 1 vacante activa, búsquedas limitadas (leer-only CV blur).
- Básico: N vacantes, filtros avanzados.
- Premium: vacantes destacadas, descargas CV, API access, analytics.
- Implementación: PaqueteEmpresaService controla límites, expiraciones y renovaciones; hosted service RenovaSuscripcion ya presente.

5.2 Integración de pagos (Stripe recomendado) - Por qué Stripe: buena integración global, manejo de suscripciones, webhooks y seguridad. - Pasos: 1. Registrar cuenta Stripe y obtener claves en secrets. 2. Instalar Stripe.net (NuGet) y crear IPaymentService -> StripePaymentService. 3. Manejar webhooks en /api/payments/webhook para eventos (checkout.session.completed, invoice.payment_succeeded). 4. Validar idempotencia y actualizar estado de PaqueteEmpresa y ComprobantePago. - Pruebas: usar Stripe CLI para emular webhooks.

5.3 Micropagos y créditos - Implementar créditos (tabla Credits) y endpoints para comprar créditos. - Descontar al descargar CV o contactar candidatos.

5.4 Marketplace y servicios profesionales - Marketplace donde proveedores venden servicios (revisión CV, simulacros). Implementar onboarding para proveedores y sistema de comisiones.

5.5 Programa de referidos - Tracking de referido por código y recompensa con créditos o descuentos.

6. Herramientas y stack recomendados (cómo implementar) Esta sección lista herramientas concretas, comandos y bibliotecas que recomiendo usar para cada tarea.

Infra y Storage - Azure Blob Storage - NuGet: Azure.Storage.Blobs - CLI: az storage blob upload/download - Docs: <https://learn.microsoft.com/azure/storage/blobs/> - AWS S3 (alternativa) - NuGet: AWSSDK.S3 - CLI: aws s3 cp - FFmpeg (transcodificación) - Instalar: apt-get install ffmpeg / brew install ffmpeg - Ejemplo: ffmpeg -i input.mp4 -c:v libx264 -preset fast -crf 23 -c:a aac out.mp4

Payments - Stripe - NuGet: Stripe.net - CLI: stripe login / stripe listen --forward-to localhost:5000/api/payments/webhook - Docs: <https://stripe.com/docs>

Observabilidad - Sentry (errores) - NuGet: Sentry.AspNetCore - Config: SENTRY_DSN en secrets - Application Insights - NuGet: Microsoft.ApplicationInsights.AspNetCore

Queues / Background Processing - RabbitMQ + MassTransit (o Azure Service Bus) - NuGet: MassTransit (MassTransit.RabbitMQ) - CLI: docker run --rm -it -p 5672:5672 rabbitmq:3-management - Hangfire (job dashboard) - NuGet: Hangfire.AspNetCore

Realtime - SignalR (ya usado). Para escala: Redis backplane - NuGet: Microsoft.AspNetCore.SignalR.StackExchangeRedis

NLP / Parsing CV - Python microservice (FastAPI) - Libraries: spaCy, pdfminer.six, transformers, sentence-transformers - Vector DB: Pinecone (managed), Weaviate, or Milvus

ML (recomendaciones) - Start heuristic in .NET; later use Python for model training. - Tools: scikit-learn, PyTorch, Hugging Face; export models via ONNX if needed.

Dev tooling y testing - dotnet SDK (>=7.0 recommended) - EF Core tools: dotnet tool install --global dotnet-ef - Unit tests: xUnit + Moq - E2E tests: Playwright or Selenium - API testing: Postman / Insomnia

CI/CD and containers - Docker - Dockerfile to containerize ChambeaJobs.Web - GitHub Actions - Workflow: build, test, docker build, push to registry, deploy - Kubernetes or Azure App Service for deployment

Local development utilities - ngrok or Stripe CLI for webhooks - sqlite for lightweight local DB (if want fast dev)

PDF generation (para informes) - DinkToPdf (wkhtmltopdf wrapper) or iText7 for server-side PDF generation - Pandoc if you want a local way to convert Markdown -> PDF - Command: pandoc docs/ChambeaJobs_Propuestas_CandidatoEmpresaMonetizacion.md -o ChambeaJobs_Propuestas.pdf

Editor/IDE - Visual Studio 2022/2023 (Windows) or VS Code (cross-platform) - Recommended extensions: C# (Omnisharp), GitLens, Docker, ESLint

7. Priorización y roadmap con pasos concretos Sprint 0 — Infra y seguridad (1-2 semanas)

- Tareas:
 1. Crear rama feature/storage-cloud.
 2. Añadir appsettings template y secrets: Storage:Provider, Azure/AWS credentials.
 3. Implementar AzureBlobStorageService y pruebas unitarias.
 4. Integrar Sentry y Application Insights.
 5. Añadir Dockerfile y pipeline CI básico.

Sprint 1 — Experiencia candidato esencial (2-3 semanas) - Tareas: 1. Crear entity PerfilCandidato y migración EF. 2. UI para edición de perfil y vista pública. 3. Implementar upload video básico y almacenarlo en storage. 4. Implementar parser pipeline mínimo (cola + worker simple).

Sprint 2 — Panel empresa y monetización básica (2-3 semanas) - Tareas: 1. Implementar StripePaymentService y endpoints para checkout. 2. Dashboard empresa con KPIs básicos. 3. Sistema de paquetes y renovación (usar hosted service existente como guía).

Sprint 3 — Matching + evaluación (3-4 semanas) - Tareas: 1. IRecomendacionService heurístico. 2. Banco de preguntas y pruebas cortas. 3. Integrar interview scheduling y Jitsi links.

Sprint 4 — Marketplace y escalado (4-6 semanas) - Tareas: 1. Marketplace inicial y onboarding de proveedores. 2. Worker para transcodificación y CDN. 3. Vector DB y embeddings si se decide.

8. Checklist técnico y mapeo al código existente (con snippets) 8.1 Cambiar IFileStorageService a provider configurable

- appsettings.json (ejemplo):

```
{
  "Storage": {
    "Provider": "Local", // Local | Azure | S3
    "Local": {
      "RootPath": "App_Data/uploads"
    },
    "Azure": {
      "ConnectionString": "<AZURE_CONN>",
      "Container": "chambea-files"
    }
  }
}
```

- Program.cs (snippet para DI):

```
var storageProvider = builder.Configuration["Storage:Provider"];
if (storageProvider == "Azure")
{
    builder.Services.AddScoped<IFileStorageService, AzureBlobStorageService>();
}
else if (storageProvider == "S3")
{
    builder.Services.AddScoped<IFileStorageService, S3StorageService>();
}
else
{
    builder.Services.AddScoped<IFileStorageService, LocalFileStorageService>();
}
```

8.2 Implementar AzureBlobStorageService (esqueleto)

```
public class AzureBlobStorageService : IFileStorageService
{
    private readonly BlobContainerClient _container;
    public AzureBlobStorageService(IConfiguration cfg)
    {
        var conn = cfg["Storage:Azure:ConnectionString"];
    }
}
```

```

        var containerName = cfg["Storage:Azure:Container"];
        var client = new BlobServiceClient(conn);
        _container = client.GetBlobContainerClient(containerName);
    }

    public async Task<string> SaveFileAsync(Stream fileStream, string fileName)
    {
        var blob = _container.GetBlobClient(fileName);
        await blob.UploadAsync(fileStream, overwrite: true);
        return blob.Uri.ToString();
    }

    // ... Delete, GetStream etc.
}

8.3 Stripe payment service (esqueleto)

public class StripePaymentService : IPaymentService
{
    public StripePaymentService(IConfiguration cfg)
    {
        StripeConfiguration.ApiKey = cfg["Stripe:SecretKey"];
    }

    public async Task<string> CreateCheckoutSessionAsync(decimal amount, string successUrl, string cancelUrl)
    {
        var options = new SessionCreateOptions
        {
            PaymentMethodTypes = new List<string> { "card" },
            LineItems = new List<SessionLineItemOptions>
            {
                new SessionLineItemOptions
                {
                    PriceData = new SessionLineItemPriceDataOptions
                    {
                        UnitAmount = (long)(amount * 100),
                        Currency = "usd",
                        ProductData = new SessionLineItemPriceDataProductDataOptions { Name = "Product" },
                    },
                    Quantity = 1
                }
            },
            Mode = "payment",
            SuccessUrl = successUrl,
            CancelUrl = cancelUrl
        };

        var service = new SessionService();
        var session = await service.CreateAsync(options);
    }
}

```

```

        return session.Id;
    }
}

```

8.4 Pipeline para parsing CV (arquitectura mínima) - Candidate uploads CV -> stored in Storage -> Message queued (RabbitMQ) with path -> CV Parser Worker consumes -> extracts JSON -> worker calls ChambeaJobs API endpoint /api/cvparser/result to store parsed data.

9. Comandos y scripts útiles para desarrollo y despliegue

- Restaurar y compilar:
 - dotnet restore
 - dotnet build
- Run migrations (local):
 - dotnet ef migrations add AddPerfilCandidato -p ChambeaJobs.Infrastructure -s ChambeaJobs.Web
 - dotnet ef database update -p ChambeaJobs.Infrastructure -s ChambeaJobs.Web
- Ejecutar la web app:
 - dotnet run --project ChambeaJobs.Web
- Docker build local:
 - docker build -t chambeajobs/web -f ChambeaJobs.Web/Dockerfile .
- Convertir Markdown a PDF (local):
 - pandoc docs/ChambeaJobs_Propuestas_CandidatoEmpresaMonetizacion.md -o ChambeaJobs_Propuestas.pdf

10. Estimaciones y recursos (muy aproximadas)

- Refactor IFileStorageService + migración a Blob: 3–7 días (1 dev)
- PWA + web push: 5–10 días (1 dev)
- Perfil enriquecido + parsing CV básico: 7–14 días (1–2 dev)
- Dashboard empresa + Stripe integration: 7–12 días (1–2 dev)
- Motor de recomendaciones heurístico: 7–14 días (1 dev)
- Implementación marketplace (fase 1): 10–20 días (2 dev)

11. Riesgos y mitigaciones

- Costos de almacenamiento y transcodificación: usar lifecycle policies, configurar reglas de retención y borrar videos antiguos.
- Privacidad y cumplimiento: registrar consentimiento, implementar endpoints para exportar/borrar datos personales.
- Fraude en pagos: validar webhooks y usar herramientas de prevención de fraude.
- Escalabilidad de realtime: agregar Redis backplane para SignalR y horizontales con sticky sessions si no usar backplane.

12. Anexos y referencias al código

- Archivos inspeccionados:
 - ChambeaJobs.Web/Program.cs
 - Controllers: AccountController.cs, CandidatoController.cs, EmpresaController.cs, VacanteController.cs, NotificacionesController.cs, ChatbotController.cs, SoporteController.cs, Admin*.

- Estructura de proyectos: ChambeaJobs.Application, ChambeaJobs.Infrastructure, ChambeaJobs.Domain, ChambeaJobs.Web.

Si quieres que genere ahora el PDF desde este Markdown y lo suba a la carpeta /docs/ como **ChambeaJobs_Propuestas_CandidatoEmpresaMonetizacion.pdf**, dime “sí, genera y súbelo” y lo haré en el siguiente paso. Si prefieres revisar o ampliar algún punto (por ejemplo, ejemplos de DB schema, diagramas, o plantillas de email), dime qué sección ampliar y lo actualizo antes de generar el PDF.